

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V) Dot
.Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T047	Name: Ashwin Tripathi
Class: TYIT	Batch: 1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

Aim : Create methods add(), multiply(), subtract(), divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

- Step 1 - Start.
- Step 2 - Declare a delegate with two integer parameters.
- Step 3 - Create methods:
 - Add(int a, int b)
 - Subtract(int a, int b)
 - Multiply(int a, int b)
 - Divide(int a, int b)
- Step 4 - In the Main() method:
 - Create a delegate object.
 - Assign the Add() method to the delegate and call it.
 - Assign the Subtract() method to the delegate and call it.
 - Assign the Multiply() method to the delegate and call it.
 - Assign the Divide() method to the delegate and call it.
- Step 5 - Display the results.
- Step 6 - Stop.

Code:

```
using System;
```

```
delegate void Operation(int a, int b);
```

```
class Calculator
```

```
{
```

```
public void Add(int a, int b)
{
    Console.WriteLine("Addition: " + (a + b));
}

public void Sub(int a, int b)
{
    Console.WriteLine("Subtraction: " + (a - b));
}

public void Mul(int a, int b)
{
    Console.WriteLine("Multiplication: " + (a * b));
}

public void Div(int a, int b)
{
    if (b != 0)
        Console.WriteLine("Division: " + (a / b));
    else
        Console.WriteLine("Cannot divide by zero.");
}
}

class Program {
    static void Main(string[] args)
    {
        Calculator c = new Calculator();
        Operation obj;

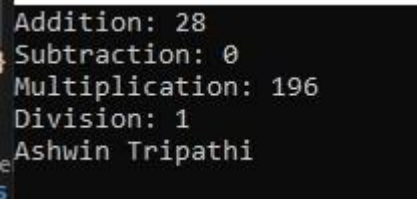
        obj = c.Add;
        obj(14, 14);

        obj = c.Sub;
        obj(14, 14);

        obj = c.Mul;
        obj(14, 14);
    }
}
```

```
obj = c.Div;  
obj(14, 14);  
  
Console.WriteLine("Ashwin Tripathi");  
}  
}
```

Output:



```
Addition: 28  
Subtraction: 0  
Multiplication: 196  
Division: 1  
Ashwin Tripathi
```

Aim :Write a program using multicast delegate.

Algorithm:

- Step 1 - Start.
- Step 2- Declare a delegate with no parameters.
- Step 3 - Create three methods (e.g., Welcome(), Learn(), ThankYou()).
- Step 4 - In the Main() method:
 - Create a delegate object.
 - Assign the first method to the delegate.
 - Add the remaining methods using the += operator.
 - Invoke the delegate object.
- Step 5 - All methods execute one after another.
- Step 6 - Stop.

Code: using
System;

```
delegate void Message();
```

```
class Demo  
{  
    public void Welcome()  
    {  
        Console.WriteLine("Welcome");  
    }  
  
    public void Learn()
```

```
{
    Console.WriteLine("Learn C#");
}

public void ThankYou()
{
    Console.WriteLine("Thank You");
}
}

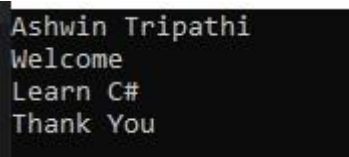
class Program {
    static void Main(string[] args)
    {
        Demo d = new Demo();
        Message m;

        m = d.Welcome;
m += d.Learn;
        m += d.ThankYou;

        Console.WriteLine("Ashwin Tripathi");

        // Invoke the multicast delegate
        m();
    }
}
```

Output:

A screenshot of a terminal window showing the output of the program. The text is displayed on four lines: "Ashwin Tripathi", "Welcome", "Learn C#", and "Thank You".

```
Ashwin Tripathi
Welcome
Learn C#
Thank You
```

Aim :Create a class BankAccount with AccountNumber and Balance.
Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

- Step 1 - Start.
- Step 2 - Create a class BankAccount.
- Step 3 - Declare private data members:

AccountNumber

Balance

Step 4 - Create a property for Balance.

Step 5 - In the set accessor:

Check whether the assigned value is greater than or equal to 0.

If yes, store the value in Balance.

Otherwise, display an error message: "Balance cannot be negative." Step 6 - Create an object of the BankAccount class in the Main() method.

Step 7 - Assign the account number.

Step 8 - Assign a valid balance and display the account details.

Step 9 - If a negative balance is entered, display the validation message. Step 10 - Stop.

Code:

using System;

class BankAcc

{

public int AccNumber { get; set; }

private double balance;

public double Balance

{

get

{

return balance;

}

set

{

if (value < 0)

{

Console.WriteLine("Balance cannot be negative.");

}

else

{

balance = value;

}

}

```
    }  
}  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        BankAcc account = new BankAcc();  
  
        account.AccNumber = 345;  
account.Balance = 1000;  
Console.WriteLine("Account Number: " +  
account.AccNumber);  
        Console.WriteLine("Balance: " + account.Balance);  
        // Attempt to set a negative balance  
account.Balance = -500;  
  
        Console.WriteLine("Balance after attempting to set negative value: " +  
account.Balance);  
  
        Console.WriteLine("Ashwin Tripathi");  
  
    }  
}
```

Output:

```
Account Number: 345  
Balance: 1000  
Balance cannot be negative.  
Balance after attempting to set negative value: 1000  
Ashwin Tripathi
```